# Generics: Bounds & Wildcards

Carsten Gips (HSBI)

**TL;DR**
Typ-Variablen können weiter eingeschränkt werden, in dem man einen verpflichtenden Ober- oder Untertyp angibt mit `extends` bzw. `super`. Damit muss der später bei der Instantiierung verwendete Typ-Parameter entweder die Oberklasse selbst sein oder davon ableiten (bei `extends`) bzw. der Typ-Parameter muss eine Oberklasse der angegebenen Schranke sein (`super`).
Durch die Einschränkung mit `extends` können in der Klasse/Methode auf der Typ-Variablen alle Methoden des angegebenen Obertyps verwendet werden.
Ein Wildcard (?) als Typ-Parameter steht für einen beliebigen Typ, wobei die Typ-Variable keinen Namen bekommt und damit innerhalb der Klasse/Methode nicht zugreifbar ist.

**Videos**

- VL Generics: Bounds & Wildcards
- Demo Wildcards

**Unterlagen**

- PDF-Version

---

# 1 Bounds: Einschränken der generischen Typen

```
public class Cps<E extends Number> {
    // Obere Schranke: E muss Number oder Subklasse sein
    // => Zugriff auf Methoden aus Number moeglich
}
Cps<Double> a;
Cps<Number> b;
Cps<String> c;  // Fehler!!!
```

- Schlüsselwort `extends` gilt hier auch für Interfaces
- Mehrere Interfaces: nach `extends` Klasse oder Interface, danach mit "&" getrennt die restlichen Interfaces:

```
class Cps<E extends KlasseOderInterface & I1 & I2 & I3> {}
```

*Anmerkung*: Der Typ-Parameter ist analog auch mit `super` (nach unten) einschränkbar

Beispiel bounds.Cps

# 2 Wildcards: Dieser Typ ist mir nicht so wichtig

Wildcard mit "?" => steht für unbestimmten Typ

```java
public class Wuppie {
    public void m1(List<?> a) { ... }
    public void m2(List<? extends Number> b) { ... }
}
```

- `m1`: `List` beliebig parametrisierbar
  => In `m1` für Objekte in Liste `a` nur Methoden von `Object` nutzbar!
- `m2`: `List` muss mit `Number` oder Subklasse parametrisiert werden.
  => Dadurch für Objekte in Liste `b` alle Methoden von `Number` nutzbar ...

Weitere Eigenschaften:

- Durch Wildcard kein Zugriff auf den Typ
- Wildcard kann durch upper bound eingeschränkt werden
- Geht nicht bei Klassen-/Interface-Definitionen

Bloch (2018): Nur für Parameter und nicht für Rückgabewerte nutzen!

# 3 Hands-On: Ausgabe für generische Listen

Ausgabe für Listen gesucht, die sowohl Elemente der Klasse A als auch Elemente der Klasse B enthalten können

```java
class A { void printInfo() { System.out.println("A"); } }
class B extends A { void printInfo() { System.out.println("B"); } }

public class X {
    public static void main(String[] args) {
        List<A> x = new ArrayList<A>();
        x.add(new A());  x.add(new B());
        printInfo(x);    // Klassenmethode in X, gesucht
        List<B> y = new ArrayList<B>();
        y.add(new B());  y.add(new B());
        printInfo(y);    // Klassenmethode in X, gesucht
    }
}
```

**Hinweis**: Dieses Beispiel beinhaltet auch Polymorphie bei/mit generischen Datentypen, bitte vorher auch das Video zum vierten Teil “Generics und Polymorphie” anschauen

## 3.1 Erster Versuch (*A* und *B* und *main()* wie oben)

```java
public class X {
    public static void printInfo(List<A> list) {
        for (A a : list) { a.printInfo(); }
    }
}
```

=> **So gehts nicht!** Eine `List<B>` ist **keine** `List<A>` (auch wenn ein `B` ein `A` ist, vgl. spätere Sitzung zu Generics und Vererbung ...)!

Beispiel wildcards.v1.X

### 3.2 Zweiter Versuch mit Wildcards (*A* und *B* und *main()* wie oben)

```
public class X {
    public static void printInfo(List<?> list) {
        for (Object a : list) { a.printInfo(); }
    }
}
```

=> **So gehts auch nicht!** Im Prinzip passt das jetzt für `List<A>` und `List<B>`. Dummerweise hat man durch das Wildcard keinen Zugriff mehr auf den Typ-Parameter und muss für den Typ der Laufvariablen in der `for`-Schleife dann `Object` nehmen. Aber `Object` kennt unser `printInfo` nicht ... Außerdem könnte man die Methode `X#printInfo` dank des Wildcards auch mit allen anderen Typen aufrufen ...

Beispiel wildcards.v2.X

### 3.3 Dritter Versuch (Lösung) mit Wildcards und Bounds (*A* und *B* und *main()* wie oben)

```
public class X {
    public static void printInfo(List<? extends A> list) {
        for (A a : list) { a.printInfo(); }
    }
}
```

Das ist die Lösung. Man erlaubt als Argument nur `List`-Objekte und fordert, dass sie mit `A` oder einer Unterklasse von `A` parametrisiert sind. D.h. in der Schleife kann man sich auf den gemeinsamen Obertyp `A` abstützen und hat dann auch wieder die `printInfo`-Methode zur Verfügung ...

Konsole wildcards.v3.X

## 4 Wrap-Up

- Ein Wildcard (`?`) als Typ-Parameter steht für einen beliebigen Typ
  - Ist in Klasse oder Methode dann aber nicht mehr zugreifbar
- Mit Bounds kann man Typ-Parameter nach oben oder nach unten einschränken (im Sinne einer Vererbungshierarchie)
  - **`extends`**: Der Typ-Parameter muss eine Unterklasse eines bestimmten Typen sein
  - **`super`**: Der Typ-Parameter muss eine Oberklasse eines bestimmten Typen sein

# 5 Zum Nachlesen

- Ullenboom (2021, Kap. 11.3)
- Oracle Corporation (2025)
- Oracle Corporation (2024)
- Bloch (2018)

---

**Lernziele**

- k3: Umgang mit Wildcards und Bounds bei generischen Klassen/Methoden

**Challenges**

**Spieler, Mannschaften und Ligen** Modellieren Sie in Java verschiedene Spielertypen sowie generische Mannschaften und Ligen, die jeweils bestimmte Spieler (-typen) bzw. Mannschaften aufnehmen können.

1. Implementieren Sie die Klasse `Spieler`, die das Interface `ISpieler` erfüllt.

```java
public interface ISpieler {
    String getName();
}
```

2. Implementieren Sie die beiden Klassen `FussballSpieler` und `BasketballSpieler` und sorgen Sie dafür, dass beide Klassen vom Compiler als Spieler betrachtet werden (geeignete Vererbungshierarchie).

3. Betrachten Sie das nicht-generische Interface `IMannschaft`. Erstellen Sie daraus ein generisches Interface `IMannschaft` mit einer Typ-Variablen. Stellen Sie durch geeignete Beschränkung der Typ-Variablen sicher, dass nur Mannschaften mit von `ISpieler` abgeleiteten Spielern gebildet werden können.

```java
public interface IMannschaft {
    boolean aufnehmen(ISpieler spieler);
    boolean rauswerfen(ISpieler spieler);
}
```

4. Betrachten Sie das nicht-generische Interface `ILiga`. Erstellen Sie daraus ein generisches Interface `ILiga` mit einer Typvariablen. Stellen Sie durch geeignete Beschränkung der Typvariablen sicher, dass nur Ligen mit von `IMannschaft` abgeleiteten Mannschaften angelegt werden können.

```java
public interface ILiga {
    boolean aufnehmen(IMannschaft mannschaft);
    boolean rauswerfen(IMannschaft mannschaft);
}
```

5. Leiten Sie von `ILiga` das **generische** Interface `IBundesLiga` ab. Stellen Sie durch geeignete Formulierung der Typvariablen sicher, dass nur Ligen mit Mannschaften angelegt werden können, deren Spieler vom Typ `FussballSpieler` (oder abgeleitet) sind.

   Realisieren Sie nun noch die Funktionalität von `IBundesLiga` als **nicht-generisches** Interface `IBundesLiga2`.

---

**Quellen**

Bloch, J. 2018. *Effective Java*. 3. Aufl. Addison-Wesley.
Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.
———. 2025. „Learn Java“. 2025. https://dev.java/learn/.
Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 09509bb (tooling: shift headings (use h1 as top-level headings) (#993), 2025-04-23)